

Weryfikacja formularza praktyk za pomocą SQL

Aplikacje internetowe 2

1 Zadanie 1. Weryfikacja formularza programu i harmonogramu praktyki za pomocą SQL

1.1 Cel zadania

Celem zadania jest testowanie integralności fragmentu bazy danych dla formularza **Program i harmonogram praktyki zawodowej** oraz sprawdzenie poprawności wprowadzonych danych za pomocą surowych zapytań SQL.

Formularz w minimalnej wersji powinien umożliwiać zapisanie:

- danych studenta,
- miejsca realizacji praktyki,
- terminu praktyki,
- wymaganej liczby dni roboczych,
- efektów kształcenia od 01 do 13,
- harmonogramu praktyki,
- podpisów osób zatwierdzających dokument.

1.2 Część A. Model ERD

Na podstawie formularza należy zaprojektować diagram ERD zawierający co najmniej następujące encje:

- studenci,
- formularze_praktyk,
- firmy,
- opiekunowie,
- efekty_ksztalcenia,
- efekty_formularza,
- harmonogram_praktyki.

Relacje powinny spełniać następujące założenia:

- jeden student może mieć wiele formularzy praktyk,
- jeden formularz dotyczy jednego studenta,
- jeden formularz dotyczy jednej firmy,
- jeden formularz może mieć wielu opiekunów,
- formularz powinien zawierać 13 efektów kształcenia,
- formularz powinien zawierać pozycje harmonogramu,
- suma dni w harmonogramie powinna wynosić 120.

1.3 Część B. Normalizacja danych

Należy zwrócić uwagę, aby baza danych była zaprojektowana zgodnie z zasadami normalizacji.

Przykłady błędnego projektu

Niepoprawnym rozwiązaniem byłoby utworzenie jednej dużej tabeli:

```
CREATE TABLE formularz_praktyki_bledny (  
    id_formularza INT PRIMARY KEY,  
    imie_studenta VARCHAR(50),  
    nazwisko_studenta VARCHAR(50),  
    nr_albumu VARCHAR(20),  
    firma VARCHAR(255),  
    efekt_01 TEXT,  
    efekt_02 TEXT,  
    efekt_03 TEXT,  
    efekt_04 TEXT,  
    efekt_05 TEXT,  
    efekt_06 TEXT,  
    efekt_07 TEXT,  
    efekt_08 TEXT,  
    efekt_09 TEXT,  
    efekt_10 TEXT,  
    efekt_11 TEXT,  
    efekt_12 TEXT,  
    efekt_13 TEXT,  
    dzial_1 VARCHAR(255),  
    dni_1 INT,  
    dzial_2 VARCHAR(255),  
    dni_2 INT  
);
```

Taki projekt jest niepoprawny, ponieważ:

- powtarza dane studenta,

- powtarza dane firmy,
- utrudnia dodawanie nowych pozycji harmonogramu,
- utrudnia wyszukiwanie efektów kształcenia,
- łamie zasady normalizacji.

Przykład poprawniejszego projektu

```
CREATE TABLE studenci (
    id_studenta INT PRIMARY KEY,
    imie VARCHAR(50) NOT NULL,
    nazwisko VARCHAR(50) NOT NULL,
    nr_albumu VARCHAR(20) NOT NULL UNIQUE,
    kierunek VARCHAR(100) NOT NULL,
    specjalnosc VARCHAR(100)
);

CREATE TABLE firmy (
    id_firmy INT PRIMARY KEY,
    nazwa VARCHAR(255) NOT NULL,
    adres VARCHAR(255)
);

CREATE TABLE opiekunowie (
    id_opiekuna INT PRIMARY KEY,
    imie VARCHAR(50) NOT NULL,
    nazwisko VARCHAR(50) NOT NULL,
    typ_opiekuna VARCHAR(30) NOT NULL
    CHECK (typ_opiekuna IN ('uczelniany', 'zakladowy'))
);

CREATE TABLE formularze_praktyk (
    id_formularza INT PRIMARY KEY,
    id_studenta INT NOT NULL,
    id_firmy INT NOT NULL,
    data_od DATE NOT NULL,
    data_do DATE NOT NULL,
    liczba_dni_roboczych INT NOT NULL CHECK (liczba_dni_roboczych =
        120),
    status VARCHAR(30) NOT NULL DEFAULT 'roboczy'
    CHECK (status IN ('roboczy', 'zatwierdzony', 'odrzucony')),
    FOREIGN KEY (id_studenta) REFERENCES studenci(id_studenta),
    FOREIGN KEY (id_firmy) REFERENCES firmy(id_firmy),
    CHECK (data_do >= data_od)
);

CREATE TABLE formularz_opiekunowie (
    id_formularza INT NOT NULL,
    id_opiekuna INT NOT NULL,
    PRIMARY KEY (id_formularza, id_opiekuna),
```

```

        FOREIGN KEY (id_formularza) REFERENCES formularze_praktyk(
            id_formularza),
        FOREIGN KEY (id_opiekuna) REFERENCES opiekunowie(id_opiekuna)
    );

CREATE TABLE efekty_ksztalcenia (
    id_efektu INT PRIMARY KEY,
    kod VARCHAR(2) NOT NULL UNIQUE,
    opis TEXT NOT NULL
);

CREATE TABLE efekty_formularza (
    id_formularza INT NOT NULL,
    id_efektu INT NOT NULL,
    opis_prac TEXT NOT NULL,
    PRIMARY KEY (id_formularza, id_efektu),
    FOREIGN KEY (id_formularza) REFERENCES formularze_praktyk(
        id_formularza),
    FOREIGN KEY (id_efektu) REFERENCES efekty_ksztalcenia(id_efektu
    )
);

CREATE TABLE harmonogram_praktyki (
    id_harmonogramu INT PRIMARY KEY,
    id_formularza INT NOT NULL,
    lp INT NOT NULL,
    dzial_komorka VARCHAR(255) NOT NULL,
    planowana_liczba_dni INT NOT NULL CHECK (planowana_liczba_dni >
        0),
    FOREIGN KEY (id_formularza) REFERENCES formularze_praktyk(
        id_formularza),
    UNIQUE (id_formularza, lp)
);

```

1.4 Część C. Dane testowe

```

INSERT INTO studenci
(id_studenta, imie, nazwisko, nr_albumu, kierunek, specjalnosc)
VALUES
(1, 'Jan', 'Kowalski', '123456', 'Informatyka', 'Aplikacje_
internetowe');

INSERT INTO firmy
(id_firmy, nazwa, adres)
VALUES
(1, 'ABC_Software_Sp. z o.o.', 'Elblag');

INSERT INTO formularze_praktyk
(id_formularza, id_studenta, id_firmy, data_od, data_do,
    liczba_dni_roboczych)

```

```
VALUES
(1, 1, 1, '2026-07-01', '2026-12-15', 120);

INSERT INTO opiekunowie
(id_opiekuna, imie, nazwisko, typ_opiekuna)
VALUES
(1, 'Anna', 'Nowak', 'uczelniany'),
(2, 'Piotr', 'Zielinski', 'zakladowy');

INSERT INTO formularz_opiekunowie
(id_formularza, id_opiekuna)
VALUES
(1, 1),
(1, 2);
```

1.5 Część D. Testy poprawności formularza

1. Sprawdzenie kompletności danych podstawowych

```
SELECT *
FROM formularze_praktyk
WHERE id_studenta IS NULL
      OR id_firmy IS NULL
      OR data_od IS NULL
      OR data_do IS NULL
      OR liczba_dni_roboczych IS NULL;
```

Jeżeli zapytanie zwraca rekordy, formularz jest niekompletny.

2. Sprawdzenie wymaganej liczby dni praktyki

```
SELECT id_formularza, liczba_dni_roboczych
FROM formularze_praktyk
WHERE liczba_dni_roboczych <> 120;
```

3. Sprawdzenie poprawności zakresu dat

```
SELECT id_formularza, data_od, data_do
FROM formularze_praktyk
WHERE data_do < data_od;
```

4. Sprawdzenie liczby efektów kształcenia

```
SELECT id_formularza, COUNT(*) AS liczba_efektow
FROM efekty_formularza
GROUP BY id_formularza
HAVING COUNT(*) <> 13;
```

5. Sprawdzenie pustych opisów efektów

```
SELECT id_formularza, id_efektu
FROM efekty_formularza
WHERE opis_prac IS NULL
      OR TRIM(opis_prac) = '';
```

6. Sprawdzenie sumy dni w harmonogramie

```
SELECT id_formularza, SUM(planowana_liczba_dni) AS suma_dni
FROM harmonogram_praktyki
GROUP BY id_formularza
HAVING SUM(planowana_liczba_dni) <> 120;
```

7. Sprawdzenie liczby pozycji harmonogramu

```
SELECT id_formularza, COUNT(*) AS liczba_pozycji
FROM harmonogram_praktyki
GROUP BY id_formularza
HAVING COUNT(*) <> 13;
```

1.6 Część E. Przypadki brzegowe

Przypadek 1. Duplikat numeru albumu

Baza danych nie powinna pozwolić na zapisanie dwóch studentów z tym samym numerem albumu.

```
INSERT INTO studenci
(id_studenta, imie, nazwisko, nr_albumu, kierunek, specjalnosc)
VALUES
(2, 'Adam', 'Nowicki', '123456', 'Informatyka', 'Sieci_komputerowe'
);
```

Oczekiwany rezultat: baza danych powinna zgłosić błąd naruszenia ograniczenia **UNIQUE**.

Przypadek 2. Niepoprawna liczba dni praktyki

```
INSERT INTO formularze_praktyk
(id_formularza, id_studenta, id_firmy, data_od, data_do,
  liczba_dni_roboczych)
VALUES
(2, 1, 1, '2026-07-01', '2026-09-30', 80);
```

Oczekiwany rezultat: baza danych powinna zgłosić błąd, ponieważ liczba dni praktyki musi wynosić 120.

Przypadek 3. Data zakończenia wcześniejsza niż data rozpoczęcia

```
INSERT INTO formularze_praktyk
(id_formularza, id_studenta, id_firmy, data_od, data_do,
 liczba_dni_roboczych)
VALUES
(3, 1, 1, '2026-10-01', '2026-07-01', 120);
```

Oczekiwany rezultat: baza danych powinna zgłosić błąd ograniczenia CHECK.

Przypadek 4. Duplikat pozycji harmonogramu

```
INSERT INTO harmonogram_praktyki
(id_harmonogramu, id_formularza, lp, dzial_komorka,
 planowana_liczba_dni)
VALUES
(1, 1, 1, 'Dzial_IT', 10);

INSERT INTO harmonogram_praktyki
(id_harmonogramu, id_formularza, lp, dzial_komorka,
 planowana_liczba_dni)
VALUES
(2, 1, 1, 'Serwis_komputerowy', 15);
```

Oczekiwany rezultat: druga instrukcja powinna zostać odrzucona, ponieważ pozycja lp = 1 dla tego samego formularza już istnieje.

Przypadek 5. Pusta nazwa działu w harmonogramie

```
SELECT id_formularza, lp
FROM harmonogram_praktyki
WHERE dzial_komorka IS NULL
OR TRIM(dzial_komorka) = '';
```

Oczekiwany rezultat: zapytanie nie powinno zwrócić żadnych rekordów.

Przypadek 6. Brak opiekuna uczelnianego lub zakładowego

```
SELECT f.id_formularza
FROM formularze_praktyk f
WHERE NOT EXISTS (
    SELECT 1
    FROM formularz_opiekunowie fo
    JOIN opiekunowie o ON fo.id_opiekuna = o.id_opiekuna
    WHERE fo.id_formularza = f.id_formularza
    AND o.typ_opiekuna = 'uczelniany'
)
OR NOT EXISTS (
    SELECT 1
    FROM formularz_opiekunowie fo
    JOIN opiekunowie o ON fo.id_opiekuna = o.id_opiekuna
```

```
WHERE fo.id_formularza = f.id_formularza
      AND o.typ_opiekuna = 'zakladowy'
);
```

1.7 Część F. Zadania do wykonania przez studenta

Student powinien:

1. opracować diagram ERD dla formularza praktyki,
2. wskazać klucze główne i obce,
3. wyjaśnić, dlaczego dane zostały rozdzielone na kilka tabel,
4. utworzyć tabele w SQL,
5. przygotować poprawne dane testowe,
6. przygotować błędne dane testowe dla przypadków brzegowych,
7. uruchomić zapytania kontrolne,
8. opisać, które zapytania wykrywają błędy formularza,
9. przygotować krótkie podsumowanie poprawności formularza.

2 Zadanie 2. Walidacja wszystkich formularzy praktyk zawodowych

2.1 Cel zadania

Celem zadania jest rozszerzenie analizy z Zadania 1 na wszystkie dokumenty związane z realizacją praktyk zawodowych oraz opracowanie spójnego modelu walidacji danych w bazie danych.

Student powinien opracować sposób sprawdzania poprawności danych dla wszystkich formularzy wykorzystywanych w procesie praktyk.

W efekcie końcowym student powinien posiadać zakodowane w SQL i udokumentowane rezultaty zwrotne dla wszystkich możliwych do przewidzenia przypadków działania na danych w BD.

2.2 Zakres analizy

Analizie podlegają wszystkie formularze zgodnie z regulaminem praktyk, w minimalnym zakresie:

- Program i harmonogram praktyki zawodowej,
- Karta praktyki zawodowej (skierowanie, oceny, potwierdzenia),

- Potwierdzenie uzyskania efektów uczenia się,
- Dziennik praktyki zawodowej,
- Sprawozdanie z praktyki zawodowej.

2.3 Część A. Spójność modelu danych

Student powinien:

- rozszerzyć model ERD z Zadania 1,
- uwzględnić wszystkie formularze jako elementy jednego systemu,
- wskazać powiązania między dokumentami (np. jeden student – wiele dokumentów),
- zapewnić spójność danych między formularzami.

Przykładowe zależności:

- dane studenta muszą być identyczne we wszystkich formularzach,
- okres praktyki musi być zgodny w każdym dokumencie,
- efekty kształcenia muszą być spójne między formularzami,
- dziennik praktyk powinien odnosić się do tych samych efektów,
- sprawozdanie powinno potwierdzać realizację efektów.

2.4 Część B. Reguły walidacji dla poszczególnych formularzy

Student powinien dla każdego formularza:

1. zidentyfikować pola obowiązkowe,
2. określić reguły poprawności danych,
3. zaproponować sposób ich walidacji w SQL (opisowo),
4. wskazać możliwe błędy.

1. Karta praktyki zawodowej

Formularz zawiera:

- dane studenta,
- skierowanie na praktykę,
- potwierdzenia (rozpoczęcie, BHP),
- oceny,
- podpisy.

Przykładowe reguły:

- brak pustych danych studenta,
- obecność obu opiekunów,
- zgodność okresu praktyki,
- oceny w zakresie 2–5,
- obecność podpisów i dat.

Walidacja w SQL (opisowo):

- sprawdzanie wartości NULL,
- sprawdzanie zakresów liczbowych,
- sprawdzanie istnienia powiązanych rekordów,
- porównanie dat.

2. Potwierdzenie efektów uczenia się

Formularz zawiera listę efektów 01–13 oraz informację, czy zostały osiągnięte.

Reguły:

- każdy efekt musi mieć przypisaną decyzję,
- nie może być jednocześnie zaznaczone „uzyskał” i „nie uzyskał”,
- liczba efektów musi wynosić 13.

Walidacja w SQL (opisowo):

- zliczanie rekordów,
- kontrola wartości logicznych,
- wykrywanie sprzecznych danych.

3. Dziennik praktyki zawodowej

Formularz zawiera wpisyienne dokumentujące przebieg praktyki.

Reguły:

- liczba wpisów powinna odpowiadać liczbie dni praktyki,
- każdy wpis musi zawierać opis pracy,
- każdy wpis musi odnosić się do efektów kształcenia,
- każdy wpis powinien być zatwierdzony podpisem.

Walidacja w SQL (opisowo):

- zliczanie wpisów,
- sprawdzanie pustych pól,
- kontrola powiązań z efektami,
- sprawdzanie podpisów.

4. Sprawozdanie z praktyki zawodowej

Formularz zawiera opis wykonanych prac i samoocenę.

Reguły:

- obecność wszystkich sekcji (charakterystyka, opis prac, kompetencje),
- zgodność opisu z dziennikiem praktyk,
- odniesienie do efektów kształcenia.

Walidacja w SQL (opisowo):

- sprawdzanie obecności danych,
- powiązanie z dziennikiem i efektami,
- analiza zgodności danych między tabelami.

2.5 Część C. Walidacja między formularzami

Student powinien zaproponować reguły sprawdzające spójność danych między dokumentami.

Przykłady:

- ten sam student we wszystkich formularzach,
- zgodność dat praktyki,
- zgodność liczby dni (120),
- zgodność efektów kształcenia,
- dziennik pokrywa cały okres praktyki,
- oceny są zgodne z osiągniętymi efektami.

Walidacja w SQL (opisowo):

- złączenia (JOIN),
- agregacje,
- porównania między tabelami.

2.6 Część D. Przypadki brzegowe

Student powinien opracować przypadki błędne dla całego systemu:

- różne daty praktyki w różnych formularzach,
- brak wpisów w dzienniku,
- brak efektów w jednym z dokumentów,
- niespójne dane studenta,

- ocena pozytywna przy braku efektów,
- brak podpisów,
- sprzeczne informacje (np. praktyka zakończona, ale brak potwierdzenia).

Dla każdego przypadku należy zaproponować sposób wykrycia w SQL.

2.7 Część E. Oczekiwany rezultat

Student powinien przygotować:

- rozszerzony model ERD,
- opis powiązań między formularzami,
- zestaw reguł walidacyjnych dla każdego dokumentu,
- zestaw reguł walidacji między dokumentami,
- propozycję zapytań SQL (opisowo),
- analizę przypadków brzegowych.